

Roteamento e Middleware

net/http – Parte 2

Prof. Fernando · DIM0547 · 2026.1

Go 1.22+: método no pattern, PathValue, padrão middleware

Roteamento no Go 1.22+

Antes (Go < 1.22): tudo era genérico

```
mux.HandleFunc("/users", func(w http.ResponseWriter, r *http.Request) {  
    switch r.Method {  
    case "GET":    listarUsuarios(w, r)  
    case "POST":   criarUsuario(w, r)  
    default:       http.Error(w, "método não permitido", 405)  
    }  
})
```

Agora (Go 1.22+): método no pattern

```
mux.HandleFunc("GET /users", listarUsuarios)  
mux.HandleFunc("POST /users", criarUsuario)  
// 405 automático para outros métodos!
```

Sem `switch`, sem `if`. O `ServeMux` resolve.

Parâmetros de rota: {nome}

```
mux.HandleFunc("GET /users/{id}", func(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id") // ← extrai o parâmetro
    fmt.Fprintf(w, "Usuário: %s", id)
})
```

Regras de precedência:

Pattern	Match	Prioridade
GET /users/{id}	/users/42	Mais específico ganha
GET /users/me	/users/me	Literal > wildcard
GET /	Qualquer GET	Catch-all (menor prioridade)

Dica: {nome...} (com ...) captura o **resto** do path: /files/{path...} casa com /files/a/b/c

Exemplo completo: CRUD

```
func main() {
    mux := http.NewServeMux()

    mux.HandleFunc("GET /contacts",    listar)
    mux.HandleFunc("POST /contacts",   criar)
    mux.HandleFunc("GET /contacts/{id}", buscar)
    mux.HandleFunc("DELETE /contacts/{id}", remover)

    http.ListenAndServe(":8080", mux)
}

func buscar(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")
    contato, ok := store[id]
    if !ok {
        http.Error(w, "não encontrado", http.StatusNotFound)
        return
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(contato)
}
```

Isso é o **Ex01 da Lista 2** — CRUD completo com ServeMux.

0 Padrão Middleware

0 que é?

Uma função que **envolve** um handler, executando código antes e/ou depois:

```
Request → [Logger] → [Auth] → [Handler] → Response
```

A assinatura

```
func MeuMiddleware(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        // ANTES: log, auth, timing, etc.  
        next.ServeHTTP(w, r)  
        // DEPOIS: log de duração, cleanup, etc.  
    })  
}
```

`http.HandlerFunc` é um **adapter** — converte uma função em `http.Handler`.

Middleware de Log

```
func Logger(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        inicio := time.Now()  
        next.ServeHTTP(w, r)  
        log.Printf("%s %s - %v", r.Method, r.URL.Path, time.Since(inicio))  
    })  
}
```

Uso:

```
mux := http.NewServeMux()  
mux.HandleFunc("GET /ping", pingHandler)  
  
// Envolve o mux inteiro:  
http.ListenAndServe(":8080", Logger(mux))
```

Problema: como capturar o **status code**? O `ResponseWriter` não expõe o status depois de `WriteHeader`. Solução: **status recorder** (Ex02 da Lista 2).

Middleware de Auth (seletivo)

```
func RequireAuth(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        token := r.Header.Get("Authorization")  
        if token != "Bearer secreto123" {  
            http.Error(w, "não autorizado", http.StatusUnauthorized)  
            return // ← NÃO chama next – interrompe a cadeia  
        }  
        next.ServeHTTP(w, r)  
    })  
}
```

Proteger apenas uma rota:

```
mux.Handle("POST /admin/config", RequireAuth(  
    http.HandlerFunc(adminConfig),  
))
```

Rotas GET continuam abertas. Só POST no admin precisa de token.

Middleware de Recovery

```
func Recovery(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        defer func() {  
            if err := recover(); err != nil {  
                log.Printf("PANIC: %v", err)  
                http.Error(w, `{"error":"internal server error"}`,  
                    http.StatusInternalServerError)  
            }  
        }()  
        next.ServeHTTP(w, r)  
    })  
}
```

Sem recovery: um `panic` derruba o servidor inteiro.

Com recovery: retorna 500, loga o erro, servidor continua rodando.

Isso é o **Ex03 da Lista 2**.

Compondo middlewares

Middlewares se compõem por aninhamento:

```
handler := Logger(Recovery(Auth(mux)))  
// Execução: Logger → Recovery → Auth → mux → Auth → Recovery → Logger
```

Função auxiliar para evitar aninhamento:

```
func Chain(h http.Handler, mws ...func(http.Handler) http.Handler) http.Handler {  
    for i := len(mws) - 1; i >= 0; i-- {  
        h = mws[i](h)  
    }  
    return h  
}  
  
// Uso:  
handler := Chain(mux, Logger, Recovery, Auth)
```

Context: passando dados entre middlewares

```
type chaveRequestID struct{}

func RequestID(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        id := fmt.Sprintf("%d", time.Now().UnixNano())
        w.Header().Set("X-Request-Id", id)

        ctx := context.WithValue(r.Context(), chaveRequestID{}, id)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

// No handler downstream:
func meuHandler(w http.ResponseWriter, r *http.Request) {
    id := r.Context().Value(chaveRequestID{}).(string)
    // usa o id...
}
```

Use **tipos privados** (`struct{}`) como chave — evita colisões entre pacotes.
Isso é o **Ex04 da Lista 2**.



Lista 2: Roteamento e Middleware

Prazo: 17/04 (sexta) — GitHub Classroom

Ex	Tema	Conceito principal
01	CRUD de contatos	<code>ServeMux</code> Go 1.22+, <code>PathValue</code> , JSON
02	Middleware de logging	<code>func(http.Handler) http.Handler</code> , status recorder
03	Middleware de recovery	<code>defer</code> , <code>recover()</code> , error handling graceful
04	Context e Request ID	<code>context.WithValue</code> , <code>r.WithContext</code> , headers

Dicas:

- Cada exercício exporta `NewRouter()` ou `NewApp()` — os testes chamam essas funções
- **Não** use `http.ListenAndServe` — testes usam `httptest` direto
- **Não** use `http.DefaultServeMux` — crie com `http.NewServeMux()`
- Leia os `*_test.go` — eles são a especificação

Próximos passos

Esta semana:

- 📝 **Lista 2** (net/http + middleware) — publicada hoje, prazo **17/04 (sex)**
- Quinta 09/04: Aula presencial — **Introdução ao Chi** + JSON em Go

Semana que vem (14-16/04):

- Terça: Chi avançado + OpenAPI
- Quinta: ● Acompanhamento online — **Sprint 1**